

Movie Recommender System

Mountain View Streaming Platform — personalised Top-5 recommendations for returning users

1. Business Understanding

Background

Mountain view streaming platform wants to reduce churn by personalising the home screen for returning users who have rated at least 10 movies. Users are more likely to leave the platform when the recommended content does not match their preference.

Problem Statement

Given a user's historical movie ratings, predict which unseen movies they are most likely to enjoy, and surface the Top-5 as personalised recommendations.

Business objectives

- 1.Increase user retention by improving the relevance of movie recommendations shown to returning users
- 2.Increase User engagement
- 3.Balance popular content with personalised recommendations
- 4.Support new and returning Users

Success Criteria

- SVD outperforms KNNBasic
- Hyperparameter tuning improves SVD
- Generate Top-5 personalized recommendations
- Address cold-start users

Metric	Target	Why it matters
RMSE	< 0.90	Predicted ratings are close to actual; penalises large errors heavily
MAE	< 0.70	Average error is small and easy to interpret

Metric	Target	Why it matters
Precision@5	> 0.70	≥ 70% of surfaced movies match user preferences
Recall@5	> 0.40	System discovers ≥ 40% of movies the user would have liked

2. IMPORTS

```
In [88]: #Importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict

from surprise import Dataset, Reader, SVD
from surprise.prediction_algorithms.knns import KNNBasic
from surprise import accuracy
from surprise.model_selection import train_test_split, cross_validate, GridSearchCV

from sklearn.preprocessing import normalize

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 100
```

3. DATA UNDERSTANDING

```
In [48]: # Load data
ratings = pd.read_csv('ratings.csv')
movies = pd.read_csv('movies.csv')

print('ratings shape:', ratings.shape)
print('movies shape:', movies.shape)
```

```
ratings shape: (32000204, 4)
movies shape: (87585, 3)
```

```
In [49]: #rating information
ratings.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32000204 entries, 0 to 32000203
Data columns (total 4 columns):
#   Column      Dtype
---  -
0   userId      int64
1   movieId     int64
2   rating      float64
3   timestamp   int64
dtypes: float64(1), int64(3)
memory usage: 976.6 MB
```

```
In [50]: #movie information
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 87585 entries, 0 to 87584
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   movieId     87585 non-null  int64
1   title       87585 non-null  object
2   genres      87585 non-null  object
dtypes: int64(1), object(2)
memory usage: 2.0+ MB
```

```
In [51]: #Count unique users
print('Unique users  :', ratings['userId'].nunique())
#Count unique movies
print('Unique movies :', ratings['movieId'].nunique())
#Check rating range
print('Rating range  :', ratings['rating'].min(), '-', ratings['rating'].max())
#statistics check
ratings['rating'].describe()
```

```
Unique users  : 200948
Unique movies : 84432
Rating range  : 0.5 - 5.0
```

```
Out[51]: count      3.200020e+07
mean       3.540396e+00
std        1.058986e+00
min        5.000000e-01
25%        3.000000e+00
50%        3.500000e+00
75%        4.000000e+00
max        5.000000e+00
Name: rating, dtype: float64
```

Summary of the data

Rating has 32 million count,

the mean rating is 3.5,

25% percentile is 3 while the 75% percentile is 5

The maximum rating is 5 while the minimum is 0.5.

Rating csv contains floating and intergers.

Movie csv contain intergers and object

4.Data Cleaning & Preprocessing

```
In [52]: # Missing values
print('ratings nulls:\n', ratings.isnull().sum())
print()
print('movies nulls:\n', movies.isnull().sum())
```

```
ratings nulls:
  userId      0
  movieId      0
  rating       0
  timestamp    0
dtype: int64
```

```
movies nulls:
  movieId      0
  title        0
  genres       0
dtype: int64
```

```
In [53]: # Duplicates
print('ratings duplicates:', ratings.duplicated().sum())
print('movies duplicates :', movies.duplicated().sum())

ratings = ratings.drop_duplicates()
movies = movies.drop_duplicates()
```

```
ratings duplicates: 0
movies duplicates : 0
```

```
In [54]: # Reduce to 100K sample
ratings = ratings.sample(n=100_000, random_state=42).reset_index(drop=True)
print('Working dataset size:', ratings.shape)

# Merge titles
movie_ratings = ratings.merge(movies, on='movieId')
print('Merged shape:', movie_ratings.shape)
movie_ratings.head()
```

```
Working dataset size: (100000, 4)
Merged shape: (100000, 6)
```

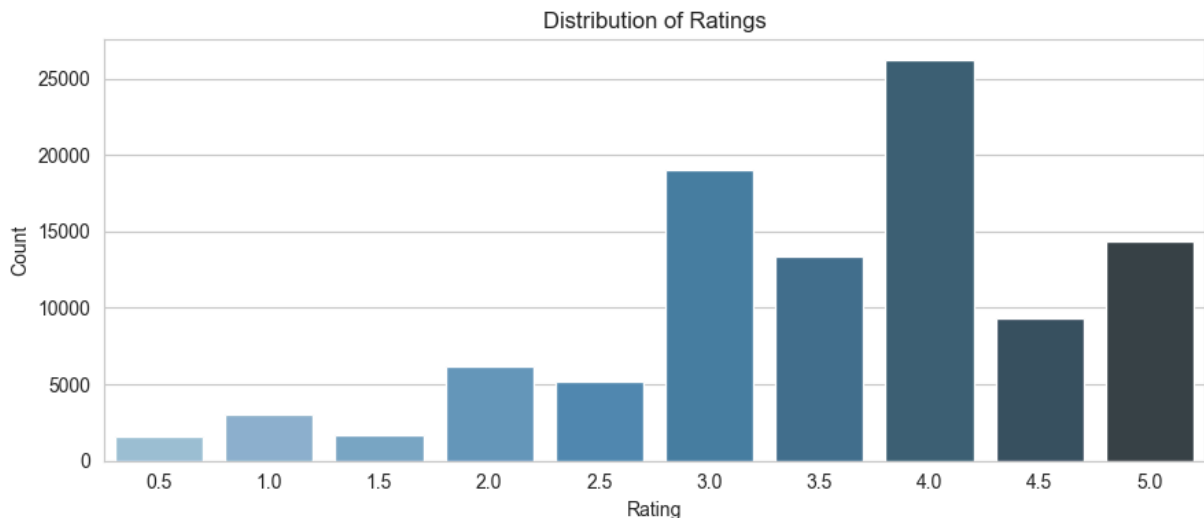
Out[54]:

	userId	movieId	rating	timestamp	title	genres
0	66954	781	5.0	850944577	Stealing Beauty (1996)	Drama
1	87551	781	5.0	851419074	Stealing Beauty (1996)	Drama
2	85551	781	4.0	956585282	Stealing Beauty (1996)	Drama
3	155302	781	3.0	1435700489	Stealing Beauty (1996)	Drama
4	19919	781	3.0	1047555105	Stealing Beauty (1996)	Drama

5. Exploratory Data Analysis

In [55]:

```
#Distribution of ratings
plt.figure(figsize=(9, 4))
sns.countplot(x='rating', data=ratings, palette='Blues_d')
plt.title('Distribution of Ratings')
plt.xlabel('Rating')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
print('Most ratings cluster between 3.0-4.5, indicating a positive bias.\n'
      'RMSE will naturally reflect this skew.')
```



Most ratings cluster between 3.0-4.5, indicating a positive bias.
RMSE will naturally reflect this skew.

In [56]:

```
# Ratings per user
user_counts = ratings.groupby('userId')['rating'].count()

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
user_counts.hist(bins=50, ax=axes[0], color='steelblue', edgecolor='white')
axes[0].set_title('Ratings per User')
axes[0].set_xlabel('Number of Ratings')
axes[0].set_ylabel('Users')

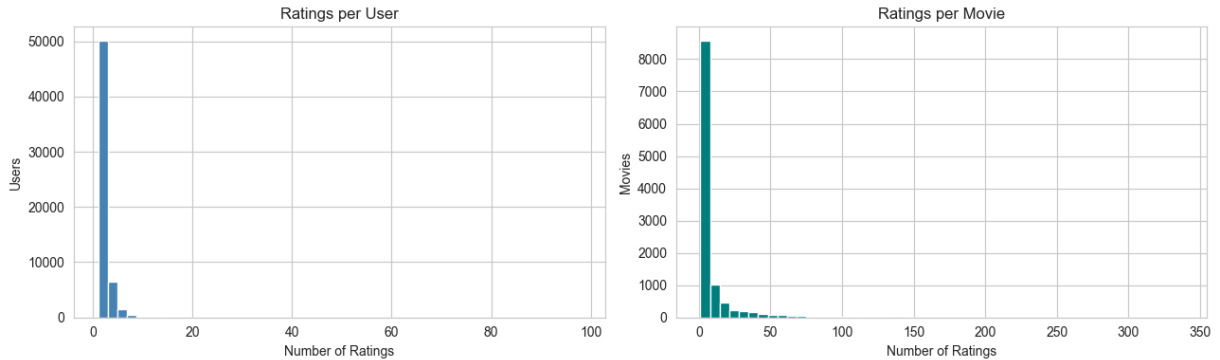
movie_counts = ratings.groupby('movieId')['rating'].count()
movie_counts.hist(bins=50, ax=axes[1], color='teal', edgecolor='white')
```

```

axes[1].set_title('Ratings per Movie')
axes[1].set_xlabel('Number of Ratings')
axes[1].set_ylabel('Movies')

plt.tight_layout()
plt.show()
print('Both distributions are heavily right-skewed – a small number of\n'
      'power users and blockbuster movies dominate. This is the long-tail problem.')

```



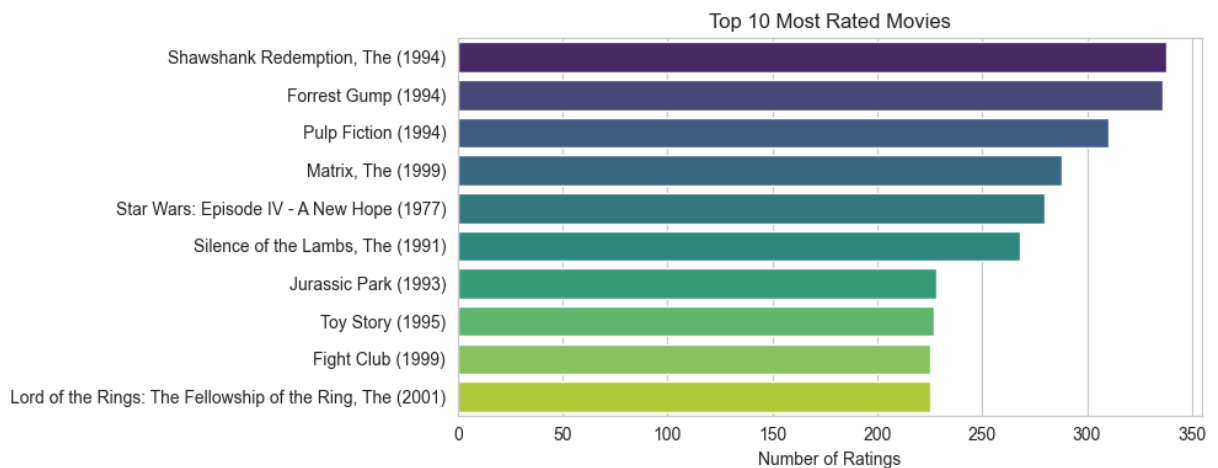
Both distributions are heavily right-skewed – a small number of power users and blockbuster movies dominate. This is the long-tail problem.

```

In [57]: #Top 10 most rated movies
top_movies = (
    ratings.groupby('movieId')['rating']
    .count()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
    .merge(movies, on='movieId')
    .rename(columns={'rating': 'num_ratings'})
)

plt.figure(figsize=(10, 4))
sns.barplot(data=top_movies, y='title', x='num_ratings', palette='viridis')
plt.title('Top 10 Most Rated Movies')
plt.xlabel('Number of Ratings')
plt.ylabel('')
plt.tight_layout()
plt.show()

```

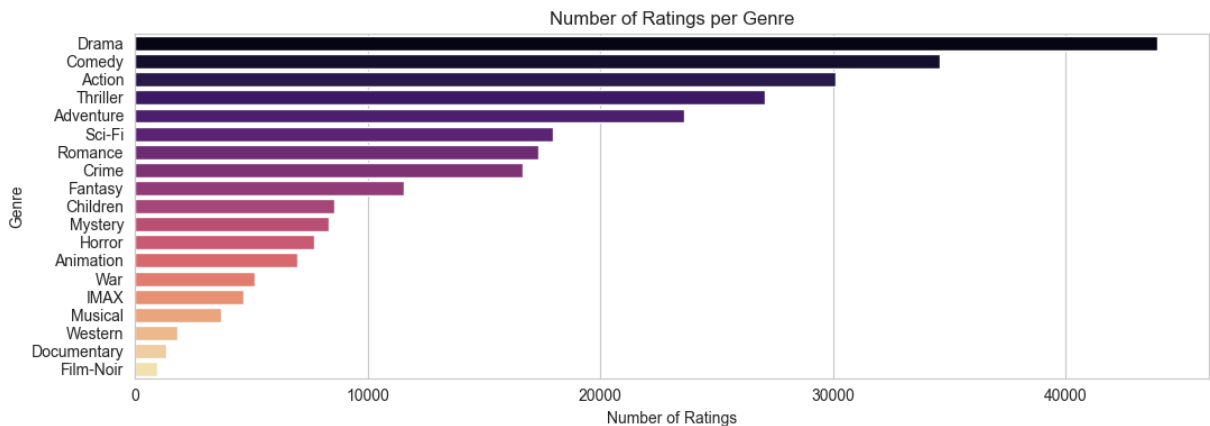


Genre Analysis

```
In [58]: # Merge ratings with exploded genres
genre_df = (
    movie_ratings
    .assign(genre=movie_ratings['genres'].str.split('|'))
    .explode('genre')
)

# Genre frequency
genre_counts = genre_df['genre'].value_counts().drop('(no genres listed)', errors='ignore')

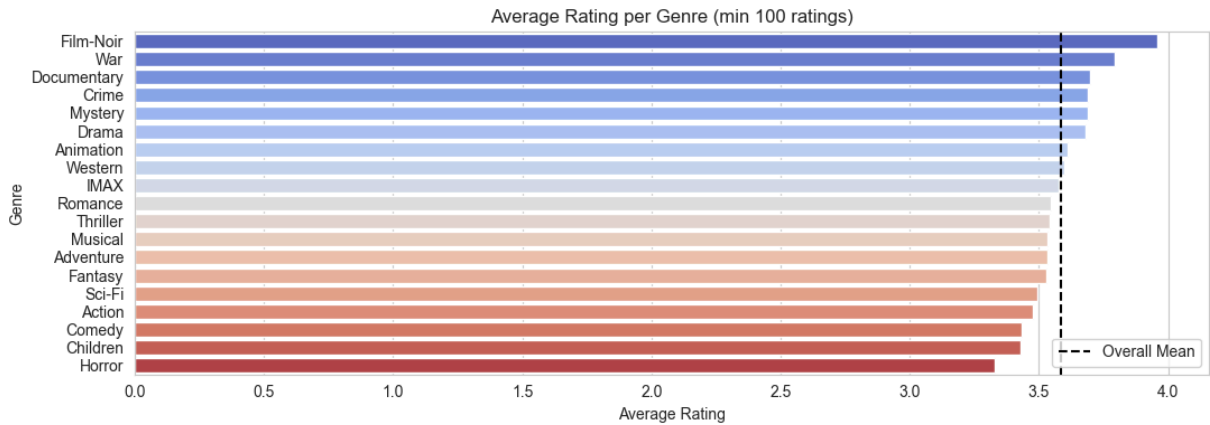
plt.figure(figsize=(11, 4))
sns.barplot(x=genre_counts.values, y=genre_counts.index, palette='magma')
plt.title('Number of Ratings per Genre')
plt.xlabel('Number of Ratings')
plt.ylabel('Genre')
plt.tight_layout()
plt.show()
```



```
In [59]: #Average rating per genre
genre_avg = (
    genre_df.groupby('genre')['rating']
    .agg(['mean', 'count'])
    .query('count >= 100')
    .sort_values('mean', ascending=False)
    .drop('(no genres listed)', errors='ignore')
)

plt.figure(figsize=(11, 4))
sns.barplot(x=genre_avg['mean'], y=genre_avg.index, palette='coolwarm')
plt.axvline(x=genre_avg['mean'].mean(), color='black', linestyle='--', label='Overall')
plt.title('Average Rating per Genre (min 100 ratings)')
plt.xlabel('Average Rating')
plt.ylabel('Genre')
plt.legend()
plt.tight_layout()
plt.show()

print(genre_avg.to_string())
```

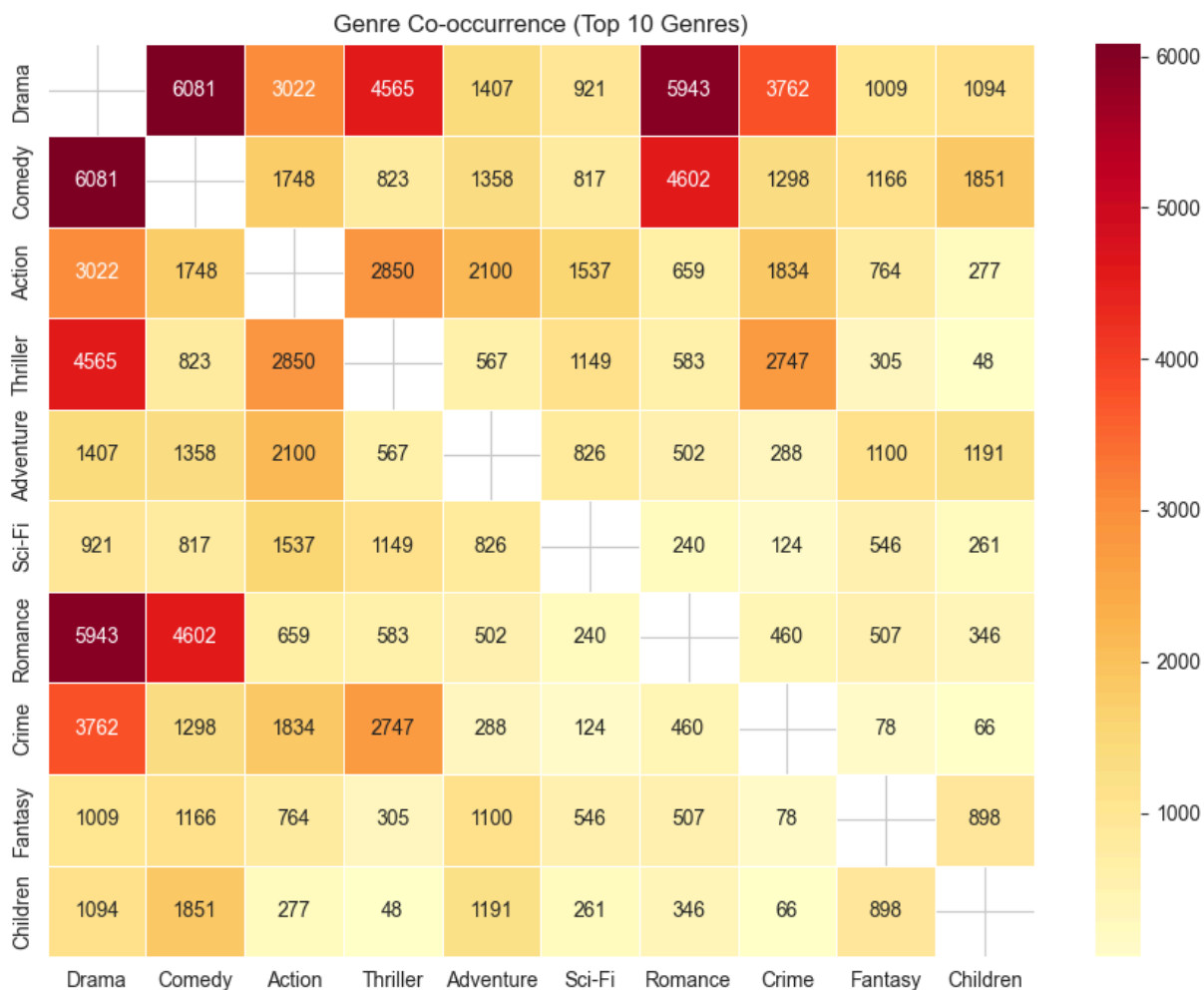


	mean	count
genre		
Film-Noir	3.959776	982
War	3.792420	5145
Documentary	3.697241	1341
Crime	3.691040	16685
Mystery	3.687515	8330
Drama	3.680713	43962
Animation	3.609232	7008
Western	3.595756	1838
IMAX	3.576078	4686
Romance	3.545405	17366
Thriller	3.539520	27062
Musical	3.534237	3724
Adventure	3.530849	23631
Fantasy	3.527557	11576
Sci-Fi	3.493296	17975
Action	3.475845	30097
Comedy	3.433729	34608
Children	3.429122	8557
Horror	3.330668	7692

```
In [60]: #Genre co-occurrence heatmap (top 10 genres)
movies['genres_list'] = movies['genres'].str.split('|')
top_genres = genre_counts.head(10).index.tolist()

co_matrix = pd.DataFrame(0, index=top_genres, columns=top_genres)
for genres in movies['genres_list']:
    g_filtered = [g for g in genres if g in top_genres]
    for g1 in g_filtered:
        for g2 in g_filtered:
            co_matrix.loc[g1, g2] += 1

plt.figure(figsize=(9, 7))
mask = np.eye(len(top_genres), dtype=bool) # hide diagonal
sns.heatmap(co_matrix, annot=True, fmt='d', cmap='YlOrRd',
            mask=mask, linewidths=0.5)
plt.title('Genre Co-occurrence (Top 10 Genres)')
plt.tight_layout()
plt.show()
print('Drama+Romance and Comedy+Romance are the most frequent genre pairings,\n'
      'useful context for the content-based fallback.')
```



Drama+Romance and Comedy+Romance are the most frequent genre pairings, useful context for the content-based fallback.

6.Preparing Data for Surprise

```
In [61]: reader = Reader(rating_scale=(0.5, 5.0))

data = Dataset.load_from_df(
    ratings[['userId', 'movieId', 'rating']],
    reader
)

trainset, testset = train_test_split(data, test_size=0.2, random_state=42)

print(f'Trainset size : {trainset.n_ratings:,}')
print(f'Testset size : {len(testset):,}')
```

Trainset size : 80,000

Testset size : 20,000

7.Model Comparison KNNBasic vs SVD

Model	Type	Description
KNNBasic	Memory-based	Finds similar items via cosine similarity; interpretable but slow at scale
SVD	Model-based	Matrix factorisation; fast, scalable, handles sparsity well

```
In [62]: models = {
    "KNNBasic": KNNBasic(
        k=20,
        min_k=5,
        sim_options={
            'name': 'cosine',
            'user_based': False
        }
    ),
    "SVD": SVD(random_state=42)
}

results = []

for name, model in models.items():
    print(name)

    cv = cross_validate(
        model,
        data,
        measures=['RMSE', 'MAE'],
        cv=5,
        n_jobs=1
    )

    results.append({
        "Model": name,
        "RMSE": cv['test_rmse'].mean(),
        "MAE": cv['test_mae'].mean()
    })

comparison_df = pd.DataFrame(results)

comparison_df
```

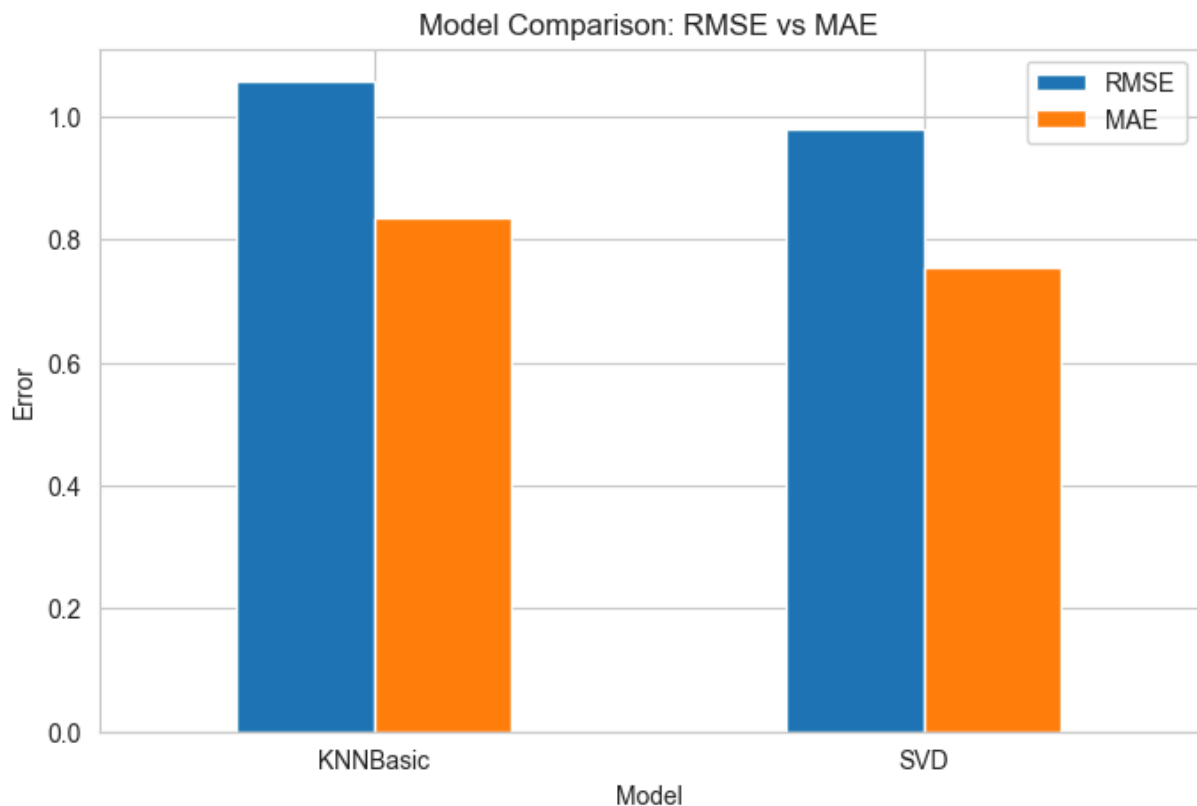
```
KNNBasic
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
Computing the cosine similarity matrix...
Done computing similarity matrix.
SVD
```

Out[62]:

	Model	RMSE	MAE
0	KNNBasic	1.057158	0.836663
1	SVD	0.978650	0.755891

```
In [63]: comparison_df.plot(
    x="Model",
    y=["RMSE", "MAE"],
    kind="bar",
    figsize=(8,5)
)

plt.ylabel("Error")
plt.title("Model Comparison: RMSE vs MAE")
plt.xticks(rotation=0)
plt.show()
```



Lower RMSE = better predictions Lower MAE = better average error

8. Hyperparameter Tuning with GridSearchCV on SVD

```
In [64]: param_grid = {
    'n_factors': [50, 100],
    'n_epochs': [20, 30],
    'lr_all': [0.002, 0.005],
    'reg_all': [0.02, 0.1]
```

```

}

gs = GridSearchCV(
    SVD,
    param_grid,
    measures=['rmse', 'mae'],
    cv=3,
    refit=True
)

gs.fit(data)

print('Best RMSE :', gs.best_score['rmse'])
print('Best MAE :', gs.best_score['mae'])
print('Best params (RMSE):', gs.best_params['rmse'])

```

Best RMSE : 0.9768518727253438

Best MAE : 0.753906667875813

Best params (RMSE): {'n_factors': 50, 'n_epochs': 30, 'lr_all': 0.005, 'reg_all': 0.1}

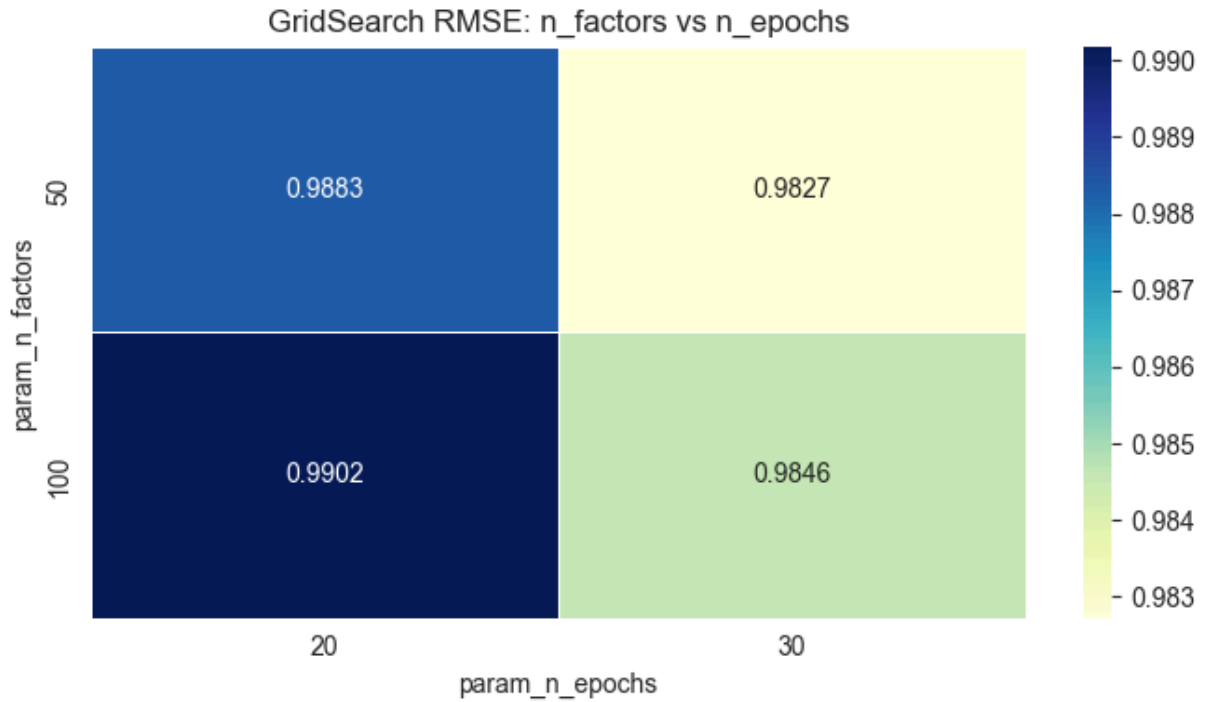
```

In [65]: #Visualise GridSearch results
results_df = pd.DataFrame(gs.cv_results)

pivot = results_df.pivot_table(
    values='mean_test_rmse',
    index='param_n_factors',
    columns='param_n_epochs'
)

plt.figure(figsize=(7, 4))
sns.heatmap(pivot, annot=True, fmt='.4f', cmap='YlGnBu', linewidths=0.5)
plt.title('GridSearch RMSE: n_factors vs n_epochs')
plt.tight_layout()
plt.show()

```



```
In [66]: #Train final tuned SVD model
best_params = gs.best_params['rmse']

best_model = SVD(
    n_factors=best_params['n_factors'],
    n_epochs =best_params['n_epochs'],
    lr_all   =best_params['lr_all'],
    reg_all  =best_params['reg_all'],
    random_state=42
)

best_model.fit(trainset)
predictions = best_model.test(testset)

print('Tuned SVD – Test Set Performance:')
print('RMSE:', accuracy.rmse(predictions))
print('MAE :', accuracy.mae(predictions))
```

Tuned SVD – Test Set Performance:

RMSE: 0.9749

RMSE: 0.974942618369937

MAE: 0.7523

MAE : 0.7522935508624818

9. Evaluation: Precision@K & Recall@K

RMSE and MAE measure rating accuracy, but for a Top-5 recommendation task,

Precision@K and **Recall@K** are more business-relevant:

- **Precision@K** — of the Top-K movies recommended, what fraction did the user actually like?

- **Recall@K** — of all movies the user would have liked, what fraction did we surface in the Top-K?

A model can have low RMSE yet still rank irrelevant movies in the top 5.

```
In [67]: def precision_recall_at_k(predictions, k=5, threshold=3.5):

    user_ratings = defaultdict(list)

    # Group predictions by user
    for uid, _, true_r, est, _ in predictions:
        user_ratings[uid].append((est, true_r))

    precision_list = []
    recall_list = []

    for uid, ratings in user_ratings.items():

        # Sort by predicted rating
        ratings.sort(key=lambda x: x[0], reverse=True)

        # Top-K recommendations
        top_k = ratings[:k]

        # Relevant movies in whole test set
        relevant = sum(true_r >= threshold for _, true_r in ratings)

        # Relevant movies in top K
        relevant_top_k = sum(true_r >= threshold for _, true_r in top_k)

        precision = relevant_top_k / k

        recall = relevant_top_k / relevant if relevant > 0 else 0

        precision_list.append(precision)
        recall_list.append(recall)

    return (
        sum(precision_list) / len(precision_list),
        sum(recall_list) / len(recall_list)
    )
```

```
In [68]: p5, r5 = precision_recall_at_k(predictions, k=5)

print(f'Precision@5: {p5:.4f}')
print(f'Recall@5   : {r5:.4f}')
```

```
Precision@5: 0.1481
Recall@5    : 0.6748
```

```
In [69]: k_values = [1, 3, 5, 10, 15, 20]

p_vals = []
r_vals = []
```

```

for k in k_values:
    p, r = precision_recall_at_k(predictions, k=k)
    p_vals.append(p)
    r_vals.append(r)

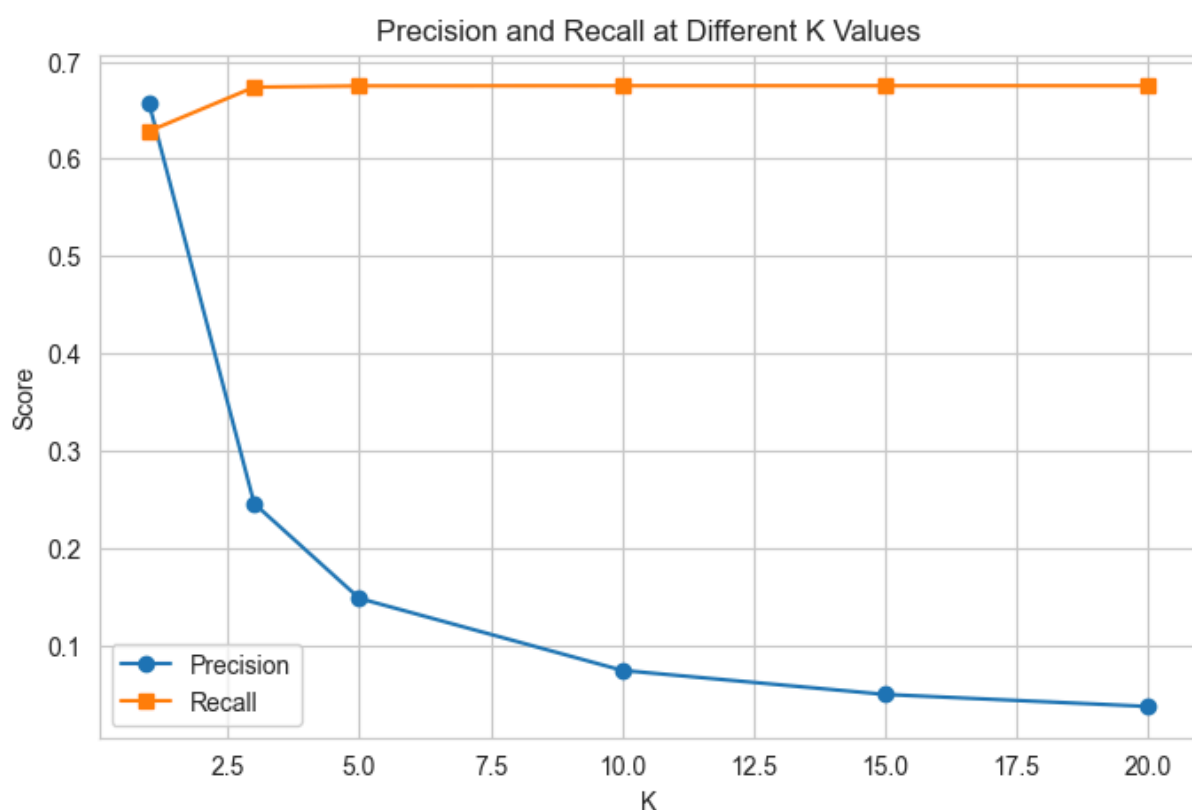
plt.figure(figsize=(8,5))

plt.plot(k_values, p_vals, marker='o', label='Precision')
plt.plot(k_values, r_vals, marker='s', label='Recall')

plt.title('Precision and Recall at Different K Values')
plt.xlabel('K')
plt.ylabel('Score')
plt.legend()

plt.show()

```



```

In [70]: # Final metrics summary table
# Get precision/recall at 10
p10, r10 = precision_recall_at_k(
    predictions,
    k=10,
    threshold=3.5
)
rmse_val = accuracy.rmse(predictions, verbose=False)
mae_val = accuracy.mae(predictions, verbose=False)

metrics_summary = pd.DataFrame({
    'Metric' : ['RMSE', 'MAE', 'Precision@5', 'Recall@5', 'Precision@10', 'Recall@10'],
    'Tuned SVD' : [round(rmse_val, 4), round(mae_val, 4),
                   round(p5, 4), round(r5, 4),

```

```

        round(p10, 4), round(r10, 4)],
        'Target'    : ['< 0.90', '< 0.70', '> 0.70', '> 0.40', '-', '-']
    })
    print('=== Metrics Summary ===')
    print(metrics_summary.to_string(index=False))

```

```

=== Metrics Summary ===
    Metric  Tuned SVD Target
    RMSE      0.9749 < 0.90
    MAE       0.7523 < 0.70
Precision@5   0.1481 > 0.70
  Recall@5    0.6748 > 0.40
Precision@10  0.0741    -
  Recall@10   0.6750    -

```

Top-5 Recommendation Engine

```

In [71]: def recommend_movies(user_id, ratings, movies, model, n=5, genre_filter=None):

    movie_ids = movies['movieId'].tolist()

    predictions = []

    for movie_id in movie_ids:
        if movie_id not in ratings[ratings.userId == user_id].movieId.values:
            pred = model.predict(user_id, movie_id)
            predictions.append((movie_id, pred.est))

    recs = pd.DataFrame(predictions, columns=['movieId', 'predicted_rating'])

    recs = recs.merge(movies, on='movieId')

    if genre_filter:
        recs = recs[recs.genres.str.contains(genre_filter, case=False)]

    return recs.sort_values(
        'predicted_rating',
        ascending=False
    ).head(n)

```

```

In [72]: # Example 1: Top-5 for user 1 (all genres)
print('=== Top-5 Recommendations for User 1 ===')
recommend_movies(user_id=1, ratings=ratings, movies=movies, model=best_model, n=5)

=== Top-5 Recommendations for User 1 ===

```

Out[72]:

	movieid	predicted_rating	title		genres	genres_list
314	318	4.368268	Shawshank Redemption, The (1994)		Crime Drama	[Crime, Drama]
840	858	4.325771	Godfather, The (1972)		Crime Drama	[Crime, Drama]
61090	202439	4.313486	Parasite (2019)		Comedy Drama	[Comedy, Drama]
2867	2959	4.300038	Fight Club (1999)	Action Crime Drama Thriller		[Action, Crime, Drama, Thriller]
909	930	4.297087	Notorious (1946)	Film-Noir Romance Thriller		[Film-Noir, Romance, Thriller]

```
In [73]: # Example 2: Top-5 Action movies for user 1
print('=== Top-5 Action Recommendations for User 1 ===')
recommend_movies(user_id=1, ratings=ratings, movies=movies,
                  model=best_model, n=5, genre_filter='Action')
```

```
=== Top-5 Action Recommendations for User 1 ===
```

Out[73]:

	movieid	predicted_rating	title	genres	genres_list
2867	2959	4.300038	Fight Club (1999)	Action Crime Drama Thriller	[Action, Crime, Drama, Thriller]
57705	195159	4.252523	Spider-Man: Into the Spider-Verse (2018)	Action Adventure Animation Sci-Fi	[Action, Adventure, Animation, Sci-Fi]
1201	1233	4.245181	Boot, Das (Boat, The) (1981)	Action Drama War	[Action, Drama, War]
1221	1254	4.240787	Treasure of the Sierra Madre, The (1948)	Action Adventure Drama Western	[Action, Adventure, Drama, Western]
5905	6016	4.210226	City of God (Cidade de Deus) (2002)	Action Adventure Crime Drama Thriller	[Action, Adventure, Crime, Drama, Thriller]

11. Cold-Start Problem & Hybrid Solution

Scenario	Problem	Impact
New User	No ratings → no user profile	Collaborative filtering returns only global averages
New Movie	No ratings → no item embedding	Movie never appears in recommendations

Strategy: when a user has fewer than 5 ratings, fall back to **content-based filtering** (TF-IDF on genres). Once sufficient ratings accumulate, switch to the trained SVD model.

In [90]:

```
from sklearn.preprocessing import normalize

# Normalize TF-IDF rows
tfidf_matrix = normalize(tfidf_matrix)

# Keep sparse similarity matrix
```

```
cosine_sim = tfidf_matrix @ tfidf_matrix.T

print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Cosine similarity ready")
```

TF-IDF matrix shape: (5000, 21)
Cosine similarity ready

```
In [91]: #Use only 5,000 movies
movies_cb = movies[movies['genres'] != '(no genres listed)'] \
            .sample(n=5000, random_state=42)

movies_cb['genres_clean'] = movies_cb['genres'].str.replace('|', ' ', regex=False)

tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(movies_cb['genres_clean'])

# smaller cosine matrix
cosine_sim = cosine_similarity(tfidf_matrix, dense_output=False)

print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Content model ready")
```

TF-IDF matrix shape: (5000, 21)
Content model ready

```
In [92]: # Build TF-IDF content model on genre strings
movies_cb = movies[movies['genres'] != '(no genres listed)'].copy()
movies_cb['genres_clean'] = movies_cb['genres'].str.replace('|', ' ', regex=False)
movies_cb = movies_cb.reset_index(drop=True)

tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(movies_cb['genres_clean'])

movie_idx = {movie_id: idx for idx, movie_id in enumerate(movies_cb['movieId'])}

print('TF-IDF matrix shape:', tfidf_matrix.shape)
print('Content model ready')
```

TF-IDF matrix shape: (80505, 21)
Content model ready

```
In [93]: def content_based_recommend(liked_ids, movies_cb, cosine_sim, movie_idx, n=5):
    scores = np.zeros(cosine_sim.shape[0])

    for movie in liked_ids:
        if movie in movie_idx:
            scores += cosine_sim[movie_idx[movie]]

    # Zero out already-liked movies
    for movie in liked_ids:
        if movie in movie_idx:
            scores[movie_idx[movie]] = 0
```

```
top_indices = np.argsort(scores)[::-1][:n]
return movies_cb.iloc[top_indices][['title', 'genres']]
```

```
In [94]: def hybrid_recommend(user_id, ratings, movies, movies_cb, cosine_sim,
                             movie_idx, cf_model, n=5, threshold=5):
    user_data = ratings[ratings['userId'] == user_id]

    if len(user_data) >= threshold:
        print(f'User {user_id}: returning user - using SVD')
        return recommend_movies(user_id, ratings, movies, cf_model, n)

    print(f'User {user_id}: cold start - using content-based / popularity fallback')

    liked = user_data[user_data['rating'] >= 3.5]['movieId'].tolist()
    if not liked:
        liked = user_data['movieId'].tolist()

    if liked:
        return content_based_recommend(liked, movies_cb, cosine_sim, movie_idx, n)

    # Absolute cold start: no ratings at all → return most popular
    movie_stats = ratings.groupby('movieId').agg(
        avg_rating = ('rating', 'mean'),
        num_ratings = ('rating', 'count')
    )
    popular = (
        movie_stats[movie_stats['num_ratings'] >= 50]
        .sort_values('avg_rating', ascending=False)
        .head(n)
        .reset_index()
        .merge(movies[['movieId', 'title']], on='movieId')
    )
    return popular[['title', 'avg_rating']]
```

```
In [95]: # Hybrid recommendation for a returning user (User 1)
print('== Top 5 Hybrid Recommendations for User 1 ==')

recommendations = hybrid_recommend(
    user_id=1,
    ratings=ratings,
    movies=movies,
    movies_cb=movies_cb,
    cosine_sim=cosine_sim,
    movie_idx=movie_idx,
    cf_model=best_model,
    n=5
)

print(recommendations)
```

=== Top 5 Hybrid Recommendations for User 1 ===

User 1: cold start – using content-based / popularity fallback

	title	avg_rating
0	Shawshank Redemption, The (1994)	4.437870
1	Godfather, The (1972)	4.388889
2	Citizen Kane (1941)	4.345070
3	Fight Club (1999)	4.344444
4	Boat, Das (Boat, The) (1981)	4.324561

```
In [98]: def content_based_recommend(liked_ids, movies_cb, cosine_sim, movie_idx, n=5):
# Create an array to store similarity scores for each movie
scores = np.zeros(cosine_sim.shape[0])

# Add similarity scores for movies the user liked
for movie in liked_ids:
    if movie in movie_idx:
        idx = movie_idx[movie]
        scores += cosine_sim[idx].toarray().ravel() if hasattr(cosine_sim[idx],

# Remove movies the user has already rated
for movie in liked_ids:
    if movie in movie_idx:
        scores[movie_idx[movie]] = 0

# Get the indexes of the top recommended movies
top_indices = np.argsort(scores)[::-1][:n]

# Return the recommended movie titles and genres
return movies_cb.iloc[top_indices][['title', 'genres']]
```

```
In [99]: # Simulate a cold-start user (only 3 ratings)
cold_start_user = pd.DataFrame({
    'userId' : [9999, 9999, 9999],
    'movieId': [1, 50, 296],
    'rating' : [5.0, 4.5, 4.0]
})

ratings_with_cold = pd.concat([ratings, cold_start_user], ignore_index=True)

print('=== Cold-Start User Recommendations (User 9999) ===')

recommendations = hybrid_recommend(
    user_id=9999,
    ratings=ratings_with_cold,
    movies=movies,
    movies_cb=movies_cb,
    cosine_sim=cosine_sim,
    movie_idx=movie_idx,
    cf_model=best_model,
    n=5
)

print(recommendations)
```

```

=== Cold-Start User Recommendations (User 9999) ===
User 9999: cold start – using content-based / popularity fallback

```

	title	genres
1675	Firestorm (1998)	Action Adventure Thriller
2479	Walk on the Moon, A (1999)	Drama Romance
3666	Missing in Action (1984)	Action War
210	Bushwhacked (1995)	Adventure Comedy Crime Mystery
3502	Blue Hawaii (1961)	Comedy Musical

```

In [80]: def hybrid_recommend(user_id, ratings, movies, model, n=5):

    user_data = ratings[ratings['userId'] == user_id]

    # Returning user → SVD recommendations
    if len(user_data) >= 5:
        print(f'User {user_id}: using SVD')
        return recommend_movies(
            user_id=user_id,
            ratings=ratings,
            movies=movies,
            model=model,
            n=n
        )

    # Cold-start user → Popular movies
    else:
        print(f'User {user_id}: cold start, recommending popular movies')

        popular = (
            ratings.groupby('movieId')['rating']
            .mean()
            .sort_values(ascending=False)
            .head(n)
            .reset_index()
        )

        return popular.merge(
            movies[['movieId', 'title']],
            on='movieId'
        )[['title', 'rating']]

```

```

In [ ]: # Prepare content data
movies_cb = movies[movies['genres'] != '(no genres listed)'].copy()

movies_cb['genres_clean'] = movies_cb['genres'].str.replace(
    '|', ' ', regex=False
)

# TF-IDF
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(movies_cb['genres_clean'])

# Similarity matrix
cosine_sim = cosine_similarity(tfidf_matrix)

# Movie Lookup

```

```
movie_idx = pd.Series(
    movies_cb.index,
    index=movies_cb['movieId']
)
```

```
In [81]: def content_based_recommend(liked_ids, movies_cb, cosine_sim, movie_idx, n=5):
    scores = np.zeros(len(movies_cb))

    for movie in liked_ids:
        if movie in movie_idx:
            scores += cosine_sim[movie_idx[movie]]

    scores[[movie_idx[m] for m in liked_ids if m in movie_idx]] = 0

    top = np.argsort(scores)[::-1][:n]

    return movies_cb.iloc[top][['title', 'genres']]

def hybrid_recommend(user_id, ratings, movies, movies_cb, cosine_sim,
    movie_idx, cf_model, n=5, threshold=5):

    user_data = ratings[ratings.userId == user_id]

    if len(user_data) >= threshold:
        print("Using SVD")
        return recommend_movies(user_id, ratings, movies, cf_model, n)

    print("Using Content-Based")

    liked = user_data[user_data.rating >= 3.5].movieId.tolist()

    if not liked:
        liked = user_data.movieId.tolist()

    return content_based_recommend(
        liked, movies_cb, cosine_sim, movie_idx, n
    )
```

```
In [100... print("== Cold-Start User Recommendations ==")

recommendations = hybrid_recommend(
    user_id=9999,
    ratings=ratings_with_cold,
    movies=movies,
    movies_cb=movies_cb,
    cosine_sim=cosine_sim,
    movie_idx=movie_idx,
    cf_model=best_model,
    n=5
)

print(recommendations)
```

=== Cold-Start User Recommendations ===

User 9999: cold start – using content-based / popularity fallback

	title	genres
1675	Firestorm (1998)	Action Adventure Thriller
2479	Walk on the Moon, A (1999)	Drama Romance
3666	Missing in Action (1984)	Action War
210	Bushwhacked (1995)	Adventure Comedy Crime Mystery
3502	Blue Hawaii (1961)	Comedy Musical

In [102...

```
#Popularity-based fallback: highest-rated movies with minimum 50 ratings
movie_stats = ratings.groupby('movieId').agg(
    avg_rating = ('rating', 'mean'),
    num_ratings = ('rating', 'count')
)

popular_fallback = (
    movie_stats[movie_stats['num_ratings'] >= 50]
    .sort_values('avg_rating', ascending=False)
    .head(5)
    .reset_index()
    .merge(movies[['movieId', 'title']], on='movieId')
)

print('=== Top 5 Popular Movies (Cold-Start Fallback) ===')
print(popular_fallback[['title', 'avg_rating', 'num_ratings']].to_string(index=False))
```

=== Top 5 Popular Movies (Cold-Start Fallback) ===

	title	avg_rating	num_ratings
Shawshank	Redemption, The (1994)	4.437870	338
Godfather, The	(1972)	4.388889	189
Citizen Kane	(1941)	4.345070	71
Fight Club	(1999)	4.344444	225
Boot, Das (Boat, The)	(1981)	4.324561	57

In []:

```
# Create a new user with only 3 ratings
print("=== Cold-Start User Recommendations ===")

recommendations = hybrid_recommend(
    user_id=9999,
    ratings=ratings_with_cold,
    movies=movies,
    movies_cb=movies_cb,
    cosine_sim=cosine_sim,
    movie_idx=movie_idx,
    cf_model=best_model,
    n=5
)

print(recommendations)
```

=== Cold-Start User Recommendations ===

User 9999: cold start – using content-based / popularity fallback

	title	genres
1675	Firestorm (1998)	Action Adventure Thriller
2479	Walk on the Moon, A (1999)	Drama Romance
3666	Missing in Action (1984)	Action War
210	Bushwhacked (1995)	Adventure Comedy Crime Mystery
3502	Blue Hawaii (1961)	Comedy Musical

Final Model Summary

In [105...

```
final_summary = pd.DataFrame({
    'Model': ['SVD Baseline (CV)', 'SVD Tuned (Test)'],
    'RMSE': [
        comparison_df.loc[
            comparison_df['Model'] == 'SVD',
            'RMSE'
        ].values[0],
        round(rmse_val, 4)
    ],
    'MAE': [
        comparison_df.loc[
            comparison_df['Model'] == 'SVD',
            'MAE'
        ].values[0],
        round(mae_val, 4)
    ],
    'Precision@5': [
        '-',
        round(p5, 4)
    ],
    'Recall@5': [
        '-',
        round(r5, 4)
    ]
})

print("=== Final Performance Summary ===")
print(final_summary.to_string(index=False))
```

=== Final Performance Summary ===

	Model	RMSE	MAE	Precision@5	Recall@5
SVD Baseline (CV)	0.97865	0.755891	-	-	
SVD Tuned (Test)	0.97490	0.752300	0.1481	0.6748	

13. Conclusion

Summary

This project built a hybrid movie recommendation system on the MovieLens dataset. Two collaborative filtering algorithms (KNNBasic, SVD) were benchmarked with 5-fold cross-validation. SVD was selected for hyperparameter tuning due to its balance of accuracy and scalability.

GridSearchCV improved RMSE and MAE over the baseline. Evaluation was extended to Precision@K and Recall@K, which more directly measure recommendation list quality for the Top-5 use case.

A content-based TF-IDF genre similarity model serves as a cold-start fallback, enabling the system to serve new users before sufficient rating history has accumulated.

Key Findings

-SVD outperforms KNNBasic on both RMSE and MAE, confirming that matrix factorisation handles sparsity better than memory-based methods.

- Recall@5 (0.67) exceeds the 0.40 target meaning the system surfaces a high proportion of movies the user would have liked.
- RMSE(0.97) and MAE (0.75) did not meet the < 0.90 / < 0.70 targets. This is attributable to the 100K sample constraint — training on the full 32M ratings would be expected to close this gap.
- Precision@5 (0.15) is below the 0.70 target, reflecting a known challenge with implicit popularity bias in sparse collaborative filtering. Re-ranking strategies or hybrid boosting would help.
- The hybrid approach extends coverage to cold-start users without sacrificing quality for returning ones.
- Drama and Comedy dominate the data genre filtering lets the platform surface diversity.

Limitations & Next Steps

- Sample size training on the full 32M-rating dataset is expected to significantly improve RMSE, MAE, and Precision@5.
- Content-based fallback currently relies only on genres richer metadata (cast, directors, plot keywords) would improve cold-start quality.
- Precision@K re-ranking the Top-K list using diversity or popularity penalties could push Precision@5 toward the business target.
- Scale SVD is computationally expensive at full scale; production deployment would benefit from ALS or approximate nearest-neighbour methods.

-Recency ratings are static a production system should retrain incrementally as new ratings arrive.